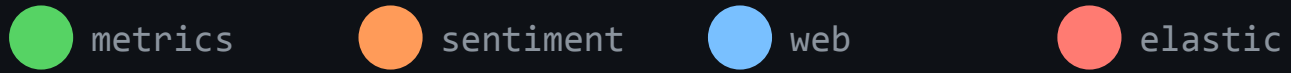
DevRel-DeepAgent

Build an Elasticsearch subagent · 2 hours



Workshop edition

The 2-hour map

Where the time goes and where you'll be coding

Half the time is hands-on.

Slides set context; the workshop happens in your editor.

Setup check

Right now, in your terminal:

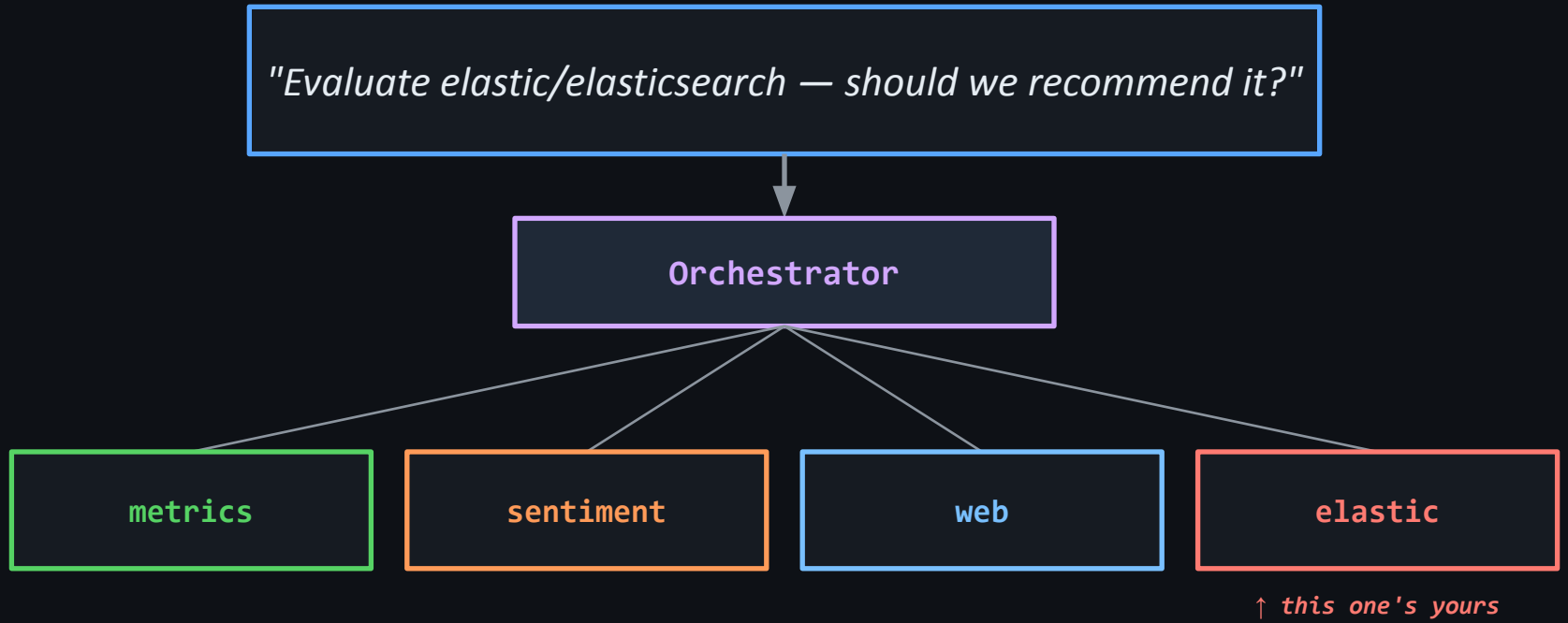
```
$ cd ~/deepagent_workshop
$ npm run verify
```

```
✓ env: LITELLM_API_KEY              set
✓ env: LITELLM_API_BASE             set
✓ ... 11 more env + service checks  set

✓ All 14 checks passed. You're ready.
```

If anything is red — raise your hand now.

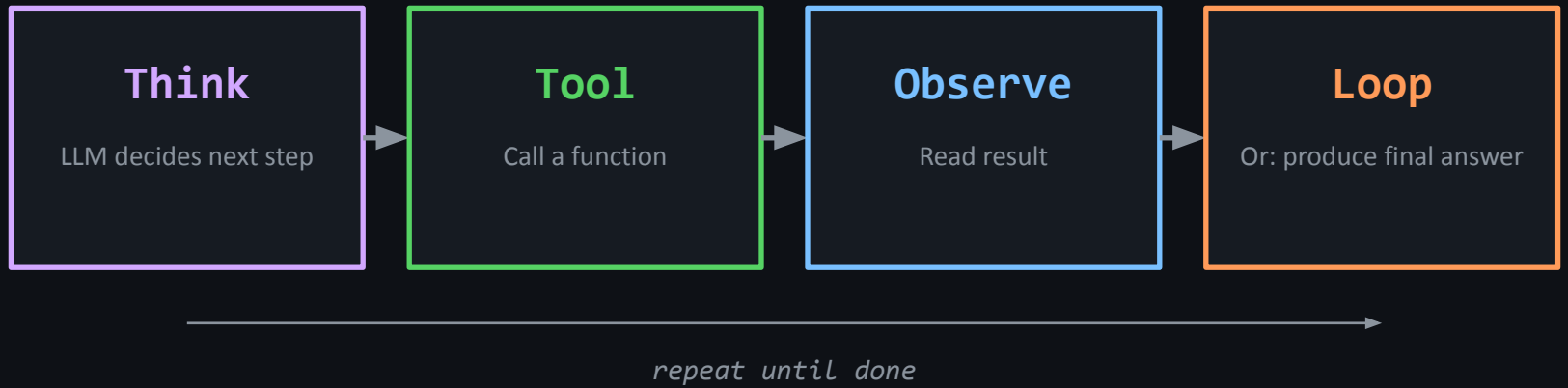
What we're building



Four specialists, one orchestrator, one final report.

The agent loop

Every agent, deep or shallow, is just this



Tools are just functions the LLM can call. The loop is just a while-loop you wrote.

Why subagents?

Three concrete reasons

Specialization

Each subagent has its own prompt, model, and tool subset. Better at its job than a generalist with 20 tools.

Context isolation

Subagent state stays in its loop. The orchestrator only sees the final reply — its context window stays clean.

Parallelism

Independent subagents fan out concurrently. Four subagents, ~3× faster than serial.

What makes an agent “deep”?

Three things, all in the deepagents package

Planner

A todo-list tool the agent uses to break a problem into steps before executing.

Subagents

Spawned via the built-in `task` tool. Each gets its own prompt, model, tools.

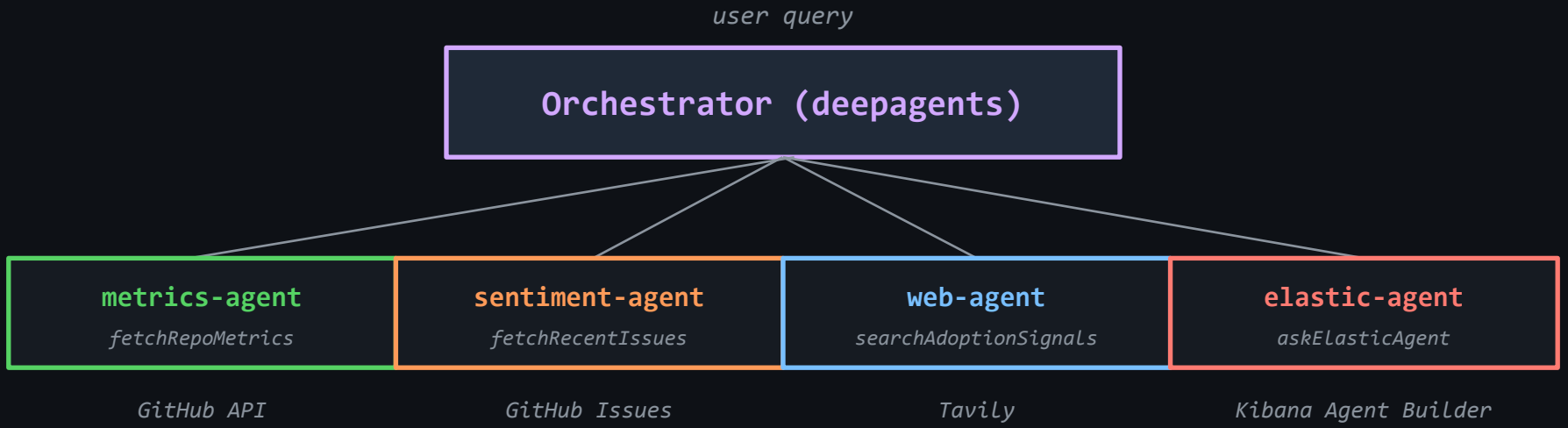
Filesystem

A virtual scratch filesystem so agents can write intermediate work and read it back.

`npm i deepagents` · github.com/langchain-ai/deepagentsjs

Today's architecture

Read top-to-bottom; data flows up at the end



↑ *each subagent returns its data* ↑ · *orchestrator synthesizes the final report*

A subagent is just an object

Four fields. That's it.

```
export const elasticSubagent = {
  name: "elastic-agent",

  description:
    "Use to retrieve cached/historical research data " +
    "from Elasticsearch via semantic search.",

  systemPrompt: elasticPrompt,    // loaded from prompts/elastic.md

  tools: [askElasticAgent],
};
```

name
unique identifier the orchestrator routes on

description
WHEN to delegate — read by orchestrator's LLM

systemPrompt
this subagent's brief — its role, tools, format

tools
what this subagent can call

What's pre-built vs what you write

Pre-built (skeleton)

- Orchestrator (src/orchestrator.ts)
- metrics-agent → GitHub repo stats
- sentiment-agent → GitHub issues
- web-agent → Tavily search
- Web UI (Express + WebSockets)
- LiteLLM-backed model factory (llm.ts)

Your code (4 files)

- `src/tools/kibanaClient.ts`
- `src/tools/askElasticAgent.ts`
- `src/subagents/elastic.ts`
- `src/prompts/elastic.md`

≈ 200 lines total

Run it — right now

See what happens before you've written anything

```
$ npm run dev

  Workshop web UI: http://localhost:3000
```

In your browser:

- metrics-agent
- sentiment-agent
- web-agent
- elastic-agent

When you click Run agent:

→ 3 cards light up and finish (metrics, sentiment, web)
→ 1 card fails — elastic-agent. That's what you'll fix.
→ Orchestrator synthesizes anyway and produces a report.

The keys problem

Why we don't ask each student for an Anthropic key

50

Attendee × 1 API key each

Account creation, billing setup, key generation — 10 min per person.

$$$

Per-attendee spend

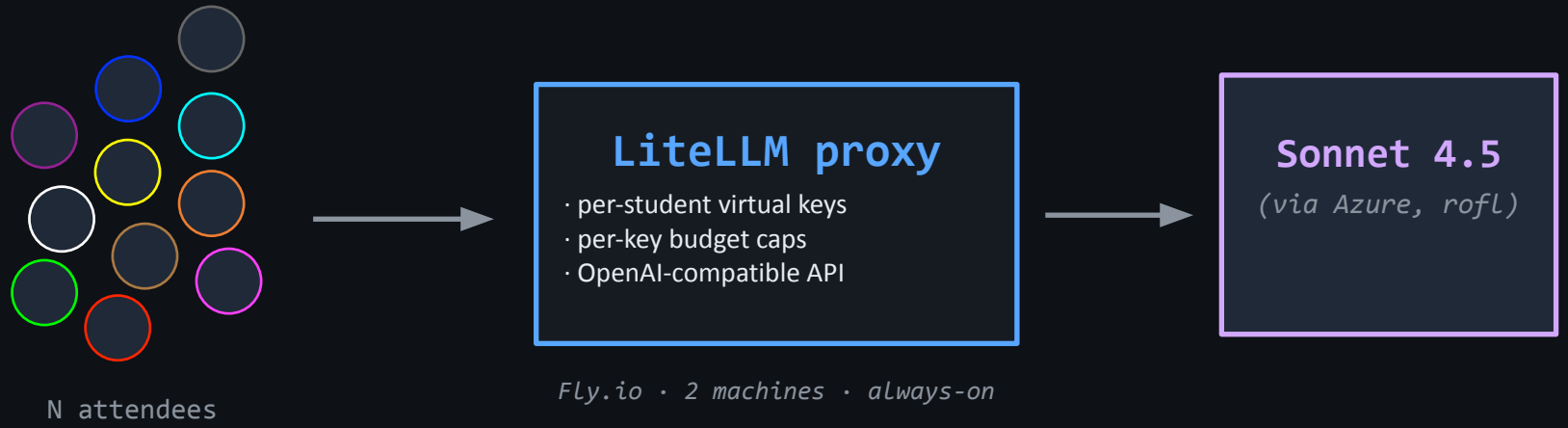
Every attendee needs ~$5 of credit. Hard for some, blocked for others.

Inconsistent setup

OpenAI vs Anthropic vs Bedrock — different SDKs, different env vars, different bugs.

The proxy solution

One proxy. One configuration. N students.



Each attendee gets a budget-capped key. Proxy translates OpenAI ↔ Anthropic transparently.

What is LiteLLM?

Open-source, OpenAI-compatible gateway for ~100 LLM providers

Drop-in OpenAI shape

- POST /v1/chat/completions
- OpenAI SDK works as-is
- Translates to provider's native format
- Translates the response back

Speaks to:

- OpenAI · Anthropic · Bedrock · Azure
- Vertex AI · Cohere · Mistral · Groq
- Ollama · vLLM · together.ai · …
- **Today: Anthropic on Azure**

github.com/BerriAI/litellm

The path of one chat completion

From your laptop to Sonnet and back

src/llm.ts — single source of truth

Every subagent calls createLLM(). One factory, one config.

```
import { ChatOpenAI } from "@langchain/openai";

export function createLLM(opts = {}) {
  return new ChatOpenAI({
    apiKey: process.env.LITELLM_API_KEY,
    configuration: { baseURL: process.env.LITELLM_API_BASE },
    model: process.env.LLM_MODEL ?? "llm-gateway/claude-sonnet-4-5",
    temperature: opts.temperature ?? 0.5,
  });
}
```

That's it. The whole LLM layer.

Swap models? Change one line. Swap providers? Change config.yaml on the proxy.

What is Elasticsearch?

Distributed JSON search + analytics engine

Inverted index

Lightning-fast full-text search over arbitrary JSON documents.

Multi-modal search

Lexical (BM25), structured filters, geospatial, vector — all in one query.

Massive scale

Sharded across nodes or server less. Used by Netflix, Uber, GitHub for log/metric/search workloads.

Today: we use it for semantic search over a research corpus.

semantic_text + ELSER

Auto-embeddings on save, auto-vector-search on query

```
// index mapping
{
  "mappings": {
    "properties": {
      "semantic_content": {
        "type": "semantic_text"
      }
    }
  }
}
```

On insert:
Elastic auto-embeds the field with ELSER (Elastic's sparse encoder). No code from you.

On query:
semantic_content : ?term
→ vector search ranked by relevance.

First write triggers ELSER auto-deploy (~30-60s). After that, it's instant.

ES|QL — pipe-syntax queries

SQL-ish, but designed for streaming pipelines

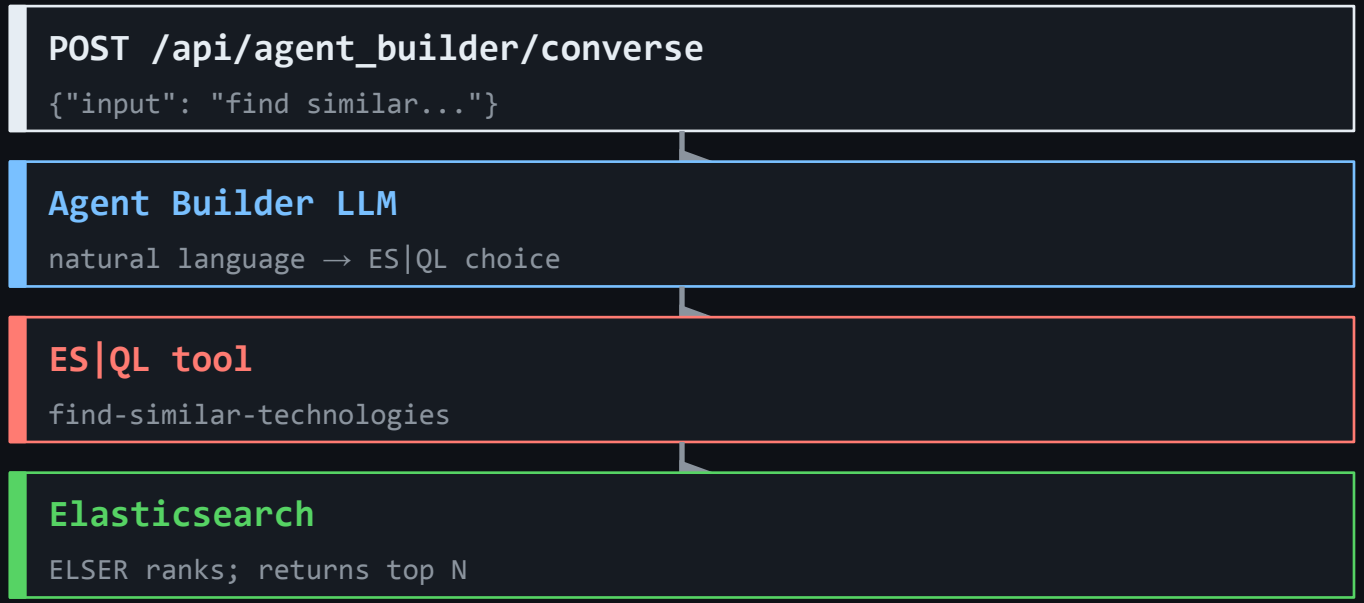
```
FROM technology-research METADATA _score
| WHERE semantic_content : ?description
| SORT _score DESC
| KEEP repo, timestamp, tags, analysis.viability_score, analysis.summary, _score
| LIMIT ?limit
```

This is the actual ES|QL query our Kibana Agent Builder tool uses.

FROM	WHERE	SORT	KEEP	LIMIT
the index	field : value = semantic match	by relevance score	only return these columns	top N results

Kibana Agent Builder

Agents that wrap ES|QL tools, exposed as a REST endpoint



What we configured for you

technology-research

Index seeded with 10 curated technology entries (LangGraph, AutoGen, Elasticsearch, Loki, OpenTelemetry, …). semantic_text on semantic_content.

find-similar-technologies

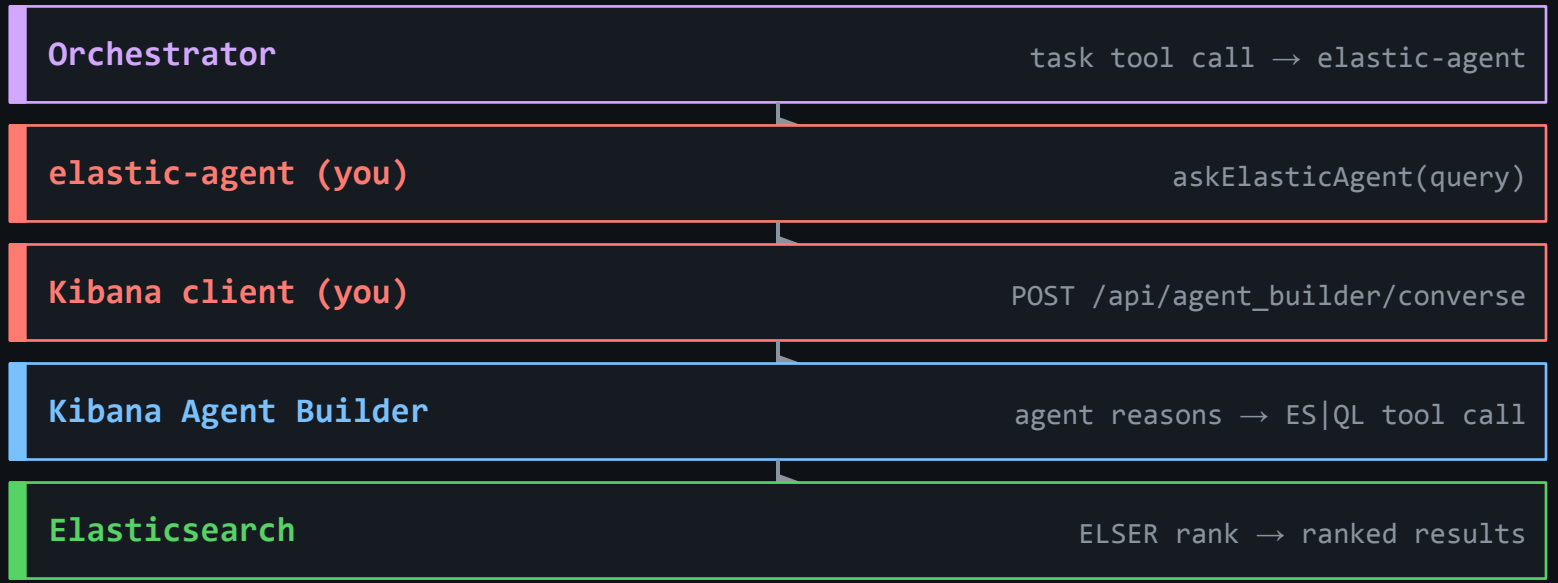
ES|QL tool registered with Kibana Agent Builder. Takes (description, limit), returns top-N matching repos.

technology-research-agent

Kibana agent attached to the tool above + a connector pointing at the LiteLLM proxy. Has an agent ID — that's your ELASTIC_AGENT_ID.

The full request path

What happens when the orchestrator dispatches elastic-agent



What you'll build, file by file

Build 1	**src/tools/kibanaClient.ts** *HTTP client*	POST /api/agent_builder/converse with auth headers; parse {response, conversation_id}
Build 2	**src/tools/askElasticAgent.ts** *LangGraph tool wrapper*	Wraps the client as a tool with a Zod schema and description
Build 3	**src/subagents/elastic.ts** *Subagent definition*	Plain object with {name, description, systemPrompt, tools}
Build 3	**src/prompts/elastic.md** *System prompt*	Markdown file the subagent's LLM reads as its instructions

Build 1 · Kibana client

src/tools/kibanaClient.ts — 4 numbered TODOs

Open: src/tools/kibanaClient.ts

TODO 1 — build the URL: <KIBANA_URL>/api/agent_builder/converse

TODO 2 — headers: Authorization, Content-Type, kbn-xsrf

TODO 3 — body: { agent_id, input, conversation_id? } ← snake_case

TODO 4 — POST + parse, return { text, conversationId }

```
# When you're done, run:
$ npm run test:client

✓ Build 1 sanity check passed.
```

Build 1 · TODOs 1, 2, & 3

```
// 1. URL
const url = `${kibanaUrl.replace(/\/$/, "")}/api/agent_builder/converse`;

// 2. Headers
const headers: Record<string, string> = {
  Authorization: `ApiKey ${apiKey}`,
  "Content-Type": "application/json",
  "kbn-xsrf": "true",
};

// 3. Body — note the snake_case keys (Kibana convention).
const body: Record<string, unknown> = {
  agent_id: req.agentId,
  input: req.input,
};
if (req.conversationId) {
  body.conversation_id = req.conversationId;
}
```

Build 1 · Todo 4

```
// 4. POST + parse.
const res = await fetch(url, {
  method: "POST",
  headers,
  body: JSON.stringify(body),
});

const data = (await res.json()) as {
  conversation_id?: string;
  response?: string | { message?: string };
};
// The `response` field may be a string OR { message: string } depending
// on the Kibana version. Handle both.
const text =
  typeof data.response === "string"
    ? data.response
    : (data.response?.message ?? "");

return {
  text,
  conversationId: data.conversation_id ?? "",
};
```

Build 2 · askElasticAgent tool

src/tools/askElasticAgent.ts — wrap the client as a LangGraph tool

Open: src/tools/askElasticAgent.ts

TODO 1 — Zod schema: { query: z.string(), conversationId?: z.string() }

TODO 2 — handler: call converse(), return its text

TODO 3 — tool description: what this tool does, when to use it

```
# When you're done, run:
$ npm run test:tool

✓ Build 2 sanity check passed.
```

Build 2 · TODO 1

```
const askElasticAgentSchema = z.object({
  query: z
    .string()
    .describe(
      "Natural-language description of what data to fetch from Elasticsearch. " +
      "Be specific about repository names, time ranges, and intent. " +
      "Examples: 'Find technologies similar to real-time observability', " +
      "'Get the latest research report for elastic/elasticsearch'.",
    ),
  conversationId: z
    .string()
    .optional()
    .describe(
      "Optional conversation ID returned by a previous askElasticAgent call. " +
      "Pass to continue a multi-turn conversation with the Elastic Agent.",
    ),
});
```

Build 2 · TODO 2

```
export const askElasticAgent = tool(
  async (input: z.infer<typeof askElasticAgentSchema>) => {
    const agentId = process.env.ELASTIC_AGENT_ID;
    if (!agentId) {
      throw new Error("Missing ELASTIC_AGENT_ID in .env");
    }

    const { text } = await converse({
      agentId,
      input: input.query,
      conversationId: input.conversationId,
    });

    return text;
  },
```

Build 2 · TODO 3

```
  {
    name: "askElasticAgent",
    description:
      "Send a natural-language request to the Elastic Agent and return its " +
      "response. The agent has access to ES|QL tools for searching, " +
      "retrieving, and analysing technology research data stored in " +
      "Elasticsearch. Use this for any data retrieval from Elasticsearch, " +
      "including: finding similar technologies via semantic search, " +
      "retrieving historical snapshots and trend data, getting adoption " +
      "signals, fetching past research reports. Be specific in queries: " +
      "include full repo names (e.g. 'elastic/elasticsearch') and time " +
      "ranges where relevant.",
    schema: askElasticAgentSchema,
  },
);
```

Build 3 · Elastic subagent + prompt

Two files. Already imported by the orchestrator.

Open: src/subagents/elastic.ts

TODO 1 — fill in the description (when to dispatch to YOU)

TODO 2 — load src/prompts/elastic.md from disk

Open: src/prompts/elastic.md

Write the system prompt: identity, mission, tool guidance, output format, honesty rule. Use the Python original as reference.

```
# When you're done, restart the dev server and ask a query:
# Ctrl+C the dev server, then:
$ npm run dev
# In your browser at localhost:3000, click "Run agent"
# The elastic-agent dot lights up red — and finishes.
```

Build 3a · TODO 1

```
const elasticPrompt = readFileSync(
  join(__dirname, "../prompts/elastic.md"),
  "utf8",
);

export const elasticSubagent = {
  name: "elastic-agent",
  description:
    "Use to retrieve cached or historical research data from Elasticsearch: " +
    "prior research reports, technology snapshots, time-series trends, " +
    "adoption signals, and similar technologies via semantic search. " +
    "Always check elastic-agent BEFORE triggering fresh data collection " +
    "from other subagents — the data may already exist.",
  systemPrompt: elasticPrompt,
  tools: [askElasticAgent],
};
```

Build 3b · TODO 2

```
You are an **Elastic Data Specialist**. You retrieve research data from
Elasticsearch by sending natural-language requests to the Elastic Agent via the
`askElasticAgent` tool.

## Your role

Gather relevant Elasticsearch data to support technology research. The Elastic
Agent handles all ES|QL query construction and execution — your job is to ask it
clearly and report the results.

## How to use askElasticAgent

Always be specific in your queries. Include:

- **Full repository names** (e.g. `elastic/elasticsearch`, not just
`elasticsearch`)
- **Time ranges** when relevant ("from the last 7 days", "last 90 days")
- **What you want** (cached report, snapshot, similar technologies, adoption
signals, trend data)

Don't try to construct ES|QL yourself — that's the Elastic Agent's job.

### Example queries

Check for a recent cached report:

> "Check if there is a cached research report for elastic/elasticsearch from the
last 7 days"

Find similar technologies via semantic search:

> "Find technologies similar to 'real-time observability and metrics monitoring',
limit 5"
```

```
Get adoption signals:

> "Get adoption signals for elastic/kibana from the last 90 days, grouped by type"

Get trend data:

> "Get trend data showing viability score changes for langchain-ai/langgraph over
6 months"

## Multi-turn conversations

If you need to follow up on a previous `askElasticAgent` call, pass the
`conversationId` returned in the previous response to maintain context.

## Output format

Structure your response clearly:

```
Elasticsearch Research Summary

Data Retrieved
- [What was found and from which repos]

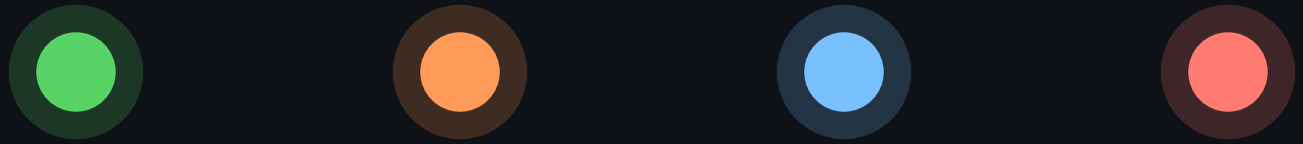
Key Findings
- [Notable data points, scores, signals]

Gaps
- [What data was missing or not found]
```
```

Run it together

Same UI as before, but now elastic works too

```
$ npm run dev
```



- ✔ *All four dots active.*
- ✔ *Activity panel scrolls.*
- ✔ *Final synthesis on the right.*

This is the deep-agent pattern in motion. You built the part that made it complete!

What you built

4 files · ~200 lines · 1 deep agent

Concepts you walked away with:

- the agent loop
- subagent orchestration via the task tool
- LLM proxying via LiteLLM (production pattern)
- semantic_text + ELSER for vector search without an embedding pipeline
- ES|QL pipe queries with parameter substitution
- Kibana Agent Builder as a NL → ES|QL gateway

Where to take it next

Add a 5th subagent	e.g. a CVE/security scanner. Same pattern: tool + prompt + register.
Persist research	Have the orchestrator write each report into the technology-research index. Later runs benefit.
Stream tokens	Pipe on_chat_model_stream events to the UI for typewriter-style output.
Skip Agent Builder	Have the elastic subagent build ES|QL itself; remove the middle LLM call.
Production hosting	Move LiteLLM behind your team's Identity Provider. Use real per-user keys.

Resources

Workshop repo	github.com/justincastilla/DevRel-DeepAgent (workshop-js branch)
deepagents JS	github.com/langchain-ai/deepagentsjs
LiteLLM	github.com/BerriAI/litellm
Elastic Agent Builder	elastic.co/docs (search Agent Builder)
ES|QL reference	elastic.co/guide/en/elasticsearch/reference/current/esql.html
semantic_text + ELSER	elastic.co/guide (search semantic_text)

Questions?

2026 Women in Data Science (WiDS) Puget Sound Conference

Women in Data Science Worldwide | Puget Sound

Event Details

DATE: May 8, 2026

FORMAT: In-person and livestream

VENUE: Mercer Island Community & Event Center

Our events are open to all, regardless of gender.

Key Features & Sessions

- Exploring Agentic AI
- Essential skills for thought leaders
- Data science for social good
- ML applications across industries

SCAN TO REGISTER